

Compilation

Analyse syntaxique

Nicolas Delestre

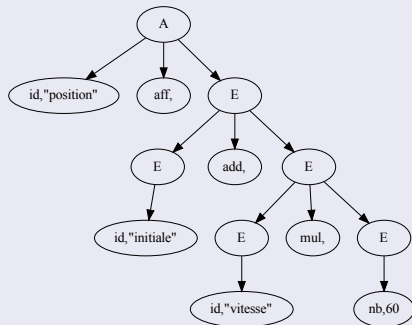
Rappels

But de l'analyse syntaxique

- Passer d'un flot de couples $\langle \text{unité lexicale}, \text{valeur} \rangle$ (valeur optionnelle) à un arbre syntaxique abstrait (AST), par exemple, passer de :

$\langle \text{id}, \text{"position"} \rangle$
 $\langle \text{aff}, \rangle$
 $\langle \text{id}, \text{"initiale"} \rangle$
 $\langle \text{add}, \rangle$
 $\langle \text{id}, \text{"vitesse"} \rangle$
 $\langle \text{mul}, \rangle$
 $\langle \text{nb}, 60 \rangle$

à



Comment ?

- Ce flot d'unités lexicales forme un mot d'un certain langage (les différentes unités lexicales sont les symboles de l'alphabet de ce langage)
- Ce langage est plus complexe qu'un simple motif, il a des aspects récurifs, par exemple :
 - *une expression est soit un nombre, soit un identifiant, soit une opération dont les opérandes sont des expressions*
- Ce langage n'est donc pas rationnel, il est algébrique (type 2), pour rappel, ses règles de production ont les deux caractéristiques suivantes :
 - la partie gauche ne contient qu'un seul non terminal
 - la partie droite n'a pas de contrainte (successions de terminaux et de non terminaux)
- La dérivation, le mécanisme de vérification d'appartenance d'un mot, permet de construire l'AST, on dit dans ce cas que l'on fait une *analyse syntaxique descendante* (on construit l'AST en partant de la racine)

Exemple 1 / 2

Grammaire (sans les valeurs des unités lexicales)

- Les terminaux sont les unités lexicales : *id*, *aff*, *add*, *sous*, *mul*, *div* et *nb*
- Les non terminaux sont *A* (pour affectation) et *E* (pour expression)
- L'axiome est le non terminal *A*
- Les règles de production sont :

$$\textcircled{1} \quad A \rightarrow id \text{ aff } E$$

$$\textcircled{2} \quad E \rightarrow E \text{ add } E$$

$$\textcircled{3} \quad E \rightarrow E \text{ sous } E$$

$$\textcircled{4} \quad E \rightarrow E \text{ mul } E$$

$$\textcircled{5} \quad E \rightarrow E \text{ div } E$$

$$\textcircled{6} \quad E \rightarrow (E)$$

$$\textcircled{7} \quad E \rightarrow nb$$

$$\textcircled{8} \quad E \rightarrow id$$

id aff id add id mul nb est-il un mot de ce langage ?

$A \Rightarrow_1 id \text{ aff } E \Rightarrow_2 id \text{ aff } E \text{ add } E \Rightarrow_8 id \text{ aff } id \text{ add } E \Rightarrow_4 id \text{ aff } id \text{ add } E \text{ mul } E \Rightarrow_8$
 $id \text{ aff } id \text{ add } id \text{ mul } E \Rightarrow_7 id \text{ aff } id \text{ add } id \text{ mul } nb$

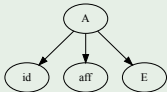
Réponse : Oui !

Exemple 2 / 2

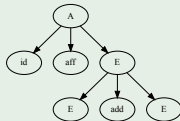
Remarque

- Cette succession de dérivations permet de construire l'arbre :

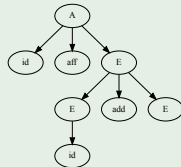
1 $A \Rightarrow_1 id\ aff\ E :$



2 $id\ aff\ E \Rightarrow_2 id\ aff\ E\ add\ E :$



3 $id\ aff\ E\ add\ E \Rightarrow_3 id\ aff\ id\ op\ E :$



4 ...

Dérivation à gauche, à droite

Définitions

- Une dérivation est dite à *gauche* (notée $\Rightarrow_g$) lorsque le premier non terminal (le plus à gauche) est remplacé par la partie de droite de la règle choisie
- Une dérivation est dite à *droite* (notée $\Rightarrow_d$) lorsque le dernier non terminal (le plus à droite) est remplacé par la partie droite de la règle choisie

Deux dérivations pour `id aff id op id op nb`

- Dérivation à gauche : $A \Rightarrow id\ aff\ E \Rightarrow id\ aff\ E\ mul\ E \Rightarrow_g id\ aff\ E\ add\ E\ mul\ E \Rightarrow_g id\ aff\ id\ add\ E\ mul\ E \Rightarrow_g id\ aff\ id\ add\ id\ mul\ E \Rightarrow id\ aff\ id\ add\ id\ mul\ nb$
- Dérivation à droite : $A \Rightarrow id\ aff\ E \Rightarrow id\ aff\ E\ mul\ E \Rightarrow_d id\ aff\ E\ mul\ nb \Rightarrow id\ aff\ E\ add\ E\ mul\ nb \Rightarrow_d id\ aff\ E\ add\ id\ mul\ nb \Rightarrow id\ aff\ id\ add\ id\ mul\ nb$

Analyseur LL

- Un analyseur LL est un analyseur syntaxique descendant qui effectue toujours des dérivations à gauche

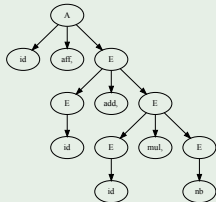
Grammaire ambiguë

Définitions

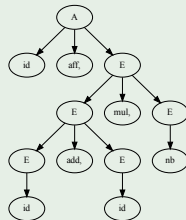
- Une grammaire LL est ambiguë lorsque, lors d'une dérivation, plusieurs règles sont possibles et que suivant le choix, l'arbre obtenu diffère

Deux dérivations possibles après $A \Rightarrow id\ aff\ E$

- $id\ aff\ E \Rightarrow id\ aff\ E\ add\ E :$



- $id\ aff\ E \Rightarrow id\ aff\ E\ mul\ E :$



« Désambiguisé » une grammaire ambiguë 1 / 4

Première méthode : réécrire la grammaire

- Cela consiste à décomposer les règles ambiguës (ayant donc la même partie gauche) en de nouvelles règles ayant des parties droites différentes

Exemple connu pour les expressions arithmétiques

- Remplacer les règles :
$$E \rightarrow E \text{ add } E \mid E \text{ sous } E \mid E \text{ mul } E \mid E \text{ div } E \mid (E) \mid id \mid nb$$
- Par les règles suivantes (T pour Terme et F pour Facteur) :
$$E \rightarrow T \mid E \text{ add } T \mid E \text{ sous } T$$
$$T \rightarrow F \mid T \text{ mul } F \mid T \text{ div } F$$
$$F \rightarrow nb \mid id \mid (E)$$
- Ainsi, après $A \Rightarrow id \text{ aff } E$ c'est uniquement la dérivation $id \text{ aff } E \Rightarrow id \text{ aff } E \text{ add } E$ qui peut être appliquée

Deuxième méthode : paramétrer l'analyseur lexicale

- Deux paramétrages possibles :
 - ① Fixer des priorités entre unités lexicales et des associativités pour une unité lexicale
 - ② Fixer le nombre k d'unités lexicales à prendre en compte pour choisir la dérivation : on parle d'analyseur LL(k)

« Désambiguisé » une grammaire ambiguë 3 / 4

Fixer des priorités entre unités lexicales et des associativités

- Cela consiste à fixer des priorités entre unités lexicales (les plus prioritaires apparaîtront plus haut dans l'AST) ou pour une unité lexicale de définir une associativité (à gauche ou à droite) lorsqu'une même unité lexicale est utilisée plusieurs fois sans des parenthèses

Exemple

- Si on spécifie que l'unité lexicale *add* est plus prioritaire que l'unité lexicale *mul* l'algorithme de l'analyseur, pour dériver *id aff E*, choisira la règle 2 ($E \rightarrow E \text{ add } E$) plutôt que la règle 4 ($E \rightarrow E \text{ mul } E$)
- Si on spécifie que l'unité lexicale est associative à gauche la suite d'unités lexicales 1 *add* 2 *add* 3 sera dérivée de la façon suivante :

$E \Rightarrow E \text{ add } E \Rightarrow 1 \text{ add } E \Rightarrow 1 \text{ add } E \text{ add } E \Rightarrow 1 \text{ add } 2 \text{ add } E \Rightarrow 1 \text{ add } 2 \text{ add } 3$

alors que si elle est définie comme associative à droite, cette même suite sera dérivée de la façon suivante :

$E \Rightarrow E \text{ add } E \Rightarrow E \text{ add } E \text{ add } E \Rightarrow 1 \text{ add } E \text{ add } E \Rightarrow 1 \text{ add } 2 \text{ add } E \Rightarrow 1 \text{ add } 2 \text{ add } 3$

Exercice

- Si l'affectation est aussi une opération, doit-on lui assigner une associativité à gauche ou à droite ?

LL(k)

- Par défaut un analyseur syntaxique LL n'a besoin que d'une seule unité lexicale pour choisir la règle à appliquer lors d'une dérivation. Mais lorsque la grammaire est ambiguë, c'est-à-dire que pour la dérivation d'un non terminal, au moins deux règles sont applicables, augmenter cette visibilité sur l'entrée de l'analyseur peut supprimer cette ambiguïté
- Si l'analyseur syntaxique a besoin de k unités lexicales, on dit que la grammaire est LL(k)
- Ce paramètre peut être appliqué globalement, pour toute la grammaire, ou pour un ensemble de règles ayant le même non terminal comme partie gauche

Réversivité à gauche

Définition

- Une grammaire est réversive à gauche lorsqu'elle possède au moins une règle du type $A \rightarrow A\alpha \mid \beta$ tel que β ne commence pas par A
- Les analyseurs LL n'accepte pas les grammaires réversives à gauche

Algorithme de suppression de la réversivité à gauche

- Remplacer $A \rightarrow A\alpha \mid \beta$ par :
 - $A \rightarrow \beta A'$
 - $A' \rightarrow \alpha A' \mid \epsilon$

Exemple : $E \rightarrow E \text{ add } T \mid E \text{ sous } T \mid T$

- Remplacé par :
 $E \rightarrow TE'$
 $E' \rightarrow \text{add } E' \mid \text{sous } E' \mid \epsilon$

Définition

- L'analyse ascendante, ou analyse LR, consiste à construire l'AST en partant des feuilles

Algorithme (très simplifié)

- Le principe est d'essayer de faire correspondre une suite d'unités lexicales avec la partie droite d'une règle :
 - Si on n'y arrive pas, on décale, c'est-à-dire que l'on prend en compte l'unité lexicale suivante ;
 - Si on y arrive, on réduit, c'est-à-dire que l'on remplace la liste d'unités lexicales représentant la partie droite d'une règle par la partie gauche (le non terminal) de cette règle

Exemple jouet

- Soit la grammaire G :

① $S \rightarrow aSbS$

② $S \rightarrow c$

- Le mot $aacbaacbcbbcbacbc$ est-il un mot de $L(G)$?

① $\underline{a}acbaacbcbbcbacbc$ on ne peut rien réduire, donc on décale

② $a\underline{a}cbaacbcbbcbacbc$ on ne peut rien réduire, donc on décale

③ $aa\underline{c}baacbcbbcbacbc$ on peut réduire $S \rightarrow c$

④ $aa\underline{S}baacbcbbcbacbc$ on ne peut rien réduire, donc on décale

⑤ $aaS\underline{b}aacbcbbcbacbc$ on ne peut rien réduire, donc on décale

⑥ $aaSb\underline{a}acbcbbcbacbc$ on ne peut rien réduire, donc on décale

⑦ $aaSba\underline{c}bcbbcbacbc$ on ne peut rien réduire, donc on décale

⑧ $aaSba\underline{c}bcbbcbacbc$ on peut réduire $S \rightarrow c$

⑨ $aaSbaa\underline{S}bcbcbacbc$ on ne peut rien réduire, donc on décale

⑩ $aaSbaa\underline{S}bcbcbacbc$ on ne peut rien réduire, donc on décale

⑪ $aaSbaa\underline{S}bcbcbacbc$ on peut réduire $S \rightarrow c$

⑫ $aaSbaa\underline{S}bcbcbacbc$ on peut réduire $S \rightarrow aSbS$

⑬ $aaSba\underline{S}bcbacbc$ on ne peut rien réduire, donc on décale

⑭ $aaSba\underline{S}bcbacbc$ on ne peut rien réduire, donc on décale

⑮ $aaSba\underline{S}bcbacbc$ on peut réduire $S \rightarrow c$

⑯ $aaSbaSb\underline{S}bacbc$ on peut réduire $S \rightarrow aSbS$

⑰ $aaSb\underline{S}bacbc$ on peut réduire $S \rightarrow aSbS$

⑱ $a\underline{S}bacbc$ on ne peut rien réduire, donc on décale

⑲ $aS\underline{b}acbc$ on ne peut rien réduire, donc on décale

⑳ $aSb\underline{a}cbc$ on ne peut rien réduire, donc on décale

㉑ $aSba\underline{c}bc$ on peut réduire $S \rightarrow c$

㉒ $aSba\underline{S}bc$ on ne peut rien réduire, donc on décale

㉓ $aSbaS\underline{b}c$ on ne peut rien réduire, donc on décale

㉔ $aSbaSb\underline{c}$ on peut réduire $S \rightarrow c$

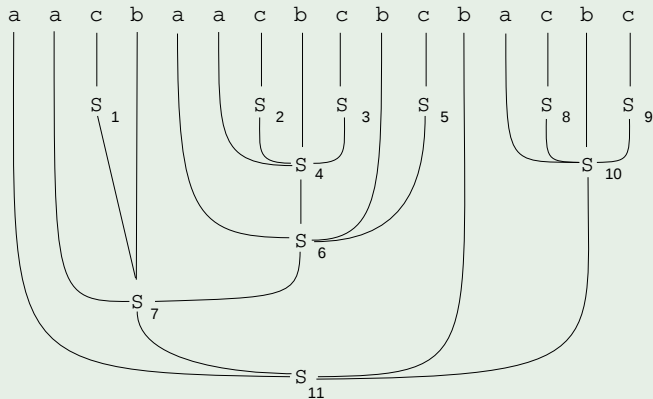
㉕ $aSbaSb\underline{S}$ on peut réduire $S \rightarrow aSbS$

㉖ $aSb\underline{S}$ on peut réduire $S \rightarrow aSbS$

㉗ S TERMINE

Analyse ascendante 3 / 3

- Ce qui donne l'AST suivant :



Conclusion

- Dans ce cours nous avons vu que :
 - les grammaires algébriques (ou non contextuelles) sont utilisées pour représenter le comportement d'un analyseur syntaxique
 - l'AST peut être obtenu par :
 - une analyse descendante (analyse LL) par le mécanisme de dérivation
 - une analyse ascendante (analyse LR) par le mécanisme de décalage, réduction
 - l'on peut « désambigüiser » une grammaire en la modifiant ou en paramétrant de l'analyseur (priorité, associativité, nombre k d'unités lexicales à lire pour choisir la règle

Un analyseur syntaxique peut être obtenu automatiquement à partir de la grammaire